

IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

RÉPUBLIQUE FRANÇAISE
Liberté Égalité Fraternité

Spatio-Temporal Transformer for Human Motion tasks

Spatio-Temporal Attention and Temporal Convolution for Motion2Text and Text2Motion Generation

Antony Manuel

IMT Nord Europe
Department of Computer Science
Master of Data Science

Lille, April 2026

Abstract

Understanding and modeling the relationship between human motion and natural language is a key challenge at the intersection of computer vision and natural language processing. This project investigates motion-language representation learning with a bidirectional focus. In the first place, it focuses on motion-to-text generation, aiming to automatically produce natural language descriptions from 3D human motion sequences. The second part of the project explores text-to-motion generation, where the goal is to synthesize realistic human motion sequences conditioned on textual descriptions. Using the HumanML3D dataset, which contains thousands of motion samples paired with rich textual annotations, I explored architectures based on transformer to encode temporal skeletal motion and decode corresponding textual descriptions, but also the other way around, then I propose in this work a novel approach with Spatio-Temporal Transformers and spatial-temporal attention mechanisms. For the first task, two complementary approaches are implemented: (1) a spatio-temporal transformer encoder coupled with a pre-trained language model decoder to leverage large-scale linguistic knowledge, and (2) a spatio-temporal transformer encoder followed by a temporal transformer decoder trained end-to-end. For the second task, a Spatio-Temporal Diffusion Transformer (DiT) is implemented to generate realistic human motion sequences from textual descriptions. Experimental results demonstrate the feasibility of generating coherent and semantically meaningful descriptions of human motion from 3D pose data. This work establishes a foundation for bidirectional motion-language modeling, with future extensions targeting gesture-to-motion generation and unified motion-language understanding frameworks. The code is available here.

Keywords: Spatio-temporal attention, Temporal Convolution, Diffusion models

Notation

$\mathcal{X}$	Set.
$\mathbb{N}$	Set of integers.
$\mathbb{R}$	Set of real numbers.
$\subset$	Strict inclusion.
$\subseteq$	Inclusion with possible equality.
$\lvert \cdot \rvert$	Absolute value of a real number.
$\lVert \cdot \rVert$	Euclidean norm of a vector.
x	A real scalar belonging to $\mathbb{R}$.
$x \in \mathbb{R}^n$	A vector in $\mathbb{R}^n$.
$X \in \mathbb{R}^{n \times d}$	A matrix in $\mathbb{R}^{n \times d}$.
I_n	Identity matrix of size n.
$X^\top$	Transpose of X.
X^{-1}	Inverse of X.
x	A random scalar variable.
$x \in \mathbb{R}^n$	A random vector in $\mathbb{R}^n$.
$X \in \mathbb{R}^{n \times d}$	A random matrix in $\mathbb{R}^{n \times d}$.
$P(x)$, $P(\mathbf{x})$	Probability distribution of x (resp. joint probability distribution of $\mathbf{x}$).
$x \sim P_x$	Random variable x follows a probability distribution P_x.
$\mathbb{E}(x)$	Expectation of x with respect to $P(x)$.
$\mathrm{Var}(x)$	Variance of x under $P(x)$.
x_i	i-th element of the vector x.
$X_{i,j}$	Element at row i, column j of the matrix X.
$X_{i,:}$	i-th row of the matrix X.
$X_{:,i}$	i-th column of the matrix X.
$f \circ g$	Composition of functions f and g.
dy/dx	Derivative of y with respect to x.
$\partial y/\partial x$	Partial derivative of y with respect to x.
$\partial f/\partial x$	Jacobian matrix $J \in \mathbb{R}^{n \times p}$ of $f : \mathbb{R}^n \to \mathbb{R}^p$.

Sometimes we use a function f whose argument is a scalar but apply it to a vector $f(x)$ or a matrix $f(X)$. This denotes the element-wise application of f to the array.

$$* \quad * \quad *$$

Contents

Contents **3**

1 Introduction **5**

2 Background **7**
1 Task Definition . . . 7
1.1 Motion Data . . . 7
1.2 Motion Captioning . . . 8
1.3 Motion Generation . . . 8
2 Attention mechanism . . . 8
3 Graph Neural Networks . . . 9
3.1 Message Passing . . . 10
4 Diffusion models . . . 11

3 Motion-to-Text **13**
1 Proposed architecture . . . 13
1.1 Overview . . . 13
1.2 Spatio-temporal Transformer . . . 13
1.2.1 Spatial Attention . . . 14
1.2.2 Temporal Convolution . . . 14
1.3 Temporal Transformer . . . 15
1.4 Transfer learning . . . 15
2 Experiments . . . 15
2.1 Loss functions . . . 16
2.2 Evaluation metrics . . . 17

4 Text-to-Motion **18**
1 Proposed architecture . . . 18
1.1 Overview . . . 18
1.2 Spatio-temporal Diffusion Transformer . . . 19
2 Experiments . . . 19

2.1 Training process 19
2.2 Training Objective 19
2.3 Evaluation metrics 20

5 Results **21**
1 Results Analysis 21
2 Discussion 21

6 Conclusion **22**

Bibliography **23**

A Appendix **26**

B Annexes **27**
1 Markov Chain 27
2 Description of Evaluation Metrics 28

Introduction

Human motion understanding is a cornerstone of human perception and cognition, as individuals continuously execute movements dictated by internal intentions and environmental interactions. Just as interpreting images, videos [1, 2], and natural language is fundamental to artificial intelligence, understanding human motion is equally critical. Consequently, modeling human kinematics has become essential across a broad spectrum of application domains, including human-computer interaction, autonomous robotics, sports analytics, healthcare diagnostics, and immersive gaming. With the development of deep learning, diverse kind of data-driven methods emerged, such as RNN methods or graph-based methods [3, 4] for motion prediction [5, 6], but also, Variational Autoencoders (VAE) [7], Generative Adversarial Networks (GAN) [8], Diffusion models [9] for realistic human motion generation.

However, modeling human motion remains a challenging task due to the complexity of human kinematics. Their inherent non-linearity, short or long-term dependencies and their inter-joints correlations make the task difficult. More importantly, human motion is mostly an unverbal communication, between humans and their environment, depending on a variety of factors such as their habits, culture, social norms and their individual intentions. Moreover, in the context of motion generation, human motion often require conditional information, such as textual descriptions or audio signals, in order to generate realistic and coherent motion sequences.

In this work, we focus, in the first place, on the problem of human motion description, which aims to generate textual descriptions of motion sequences. Secondly, we focus on the problem of human motion generation, which aims to generate motion sequences from textual descriptions. For both of these problems, the main challenge is to take into account both temporal dynamics and spatial dependencies whitin human body joints. As a mean to address this challenge, in both task we propose to consider the human body as a graph, where the joints are represented as nodes and the bones as edges. Therefore, as defined in Chen et al. [10], we propose to leverage spatio-temporal attention mechanism with temporal graph convolutions [11] and spatio-temporal graph convolutions [12] to model temporal and spatial dependencies of body joints. For the first task, we studied the effectiveness of such mechanism to extract temporal and spatial features from motion data and propose to leverage pre-trained language models such as T5 model sentence generation, as it showed

promising results in generated text [13]. Then, for the second one, we propose to leverage diffusion processes coupled with transformer architecture to generate motion sequences from textual descriptions. The proposed method is evaluated on the HumanML3D dataset[1] [14], which is a large-scale dataset of human motion sequences with corresponding textual descriptions.

This report is organized as follows: In chapter 2, we provide a background on both tasks, but also on the architecture used in this work. Chapter 3, 4, cover the proposed architecture for both motion-to-text captioning and text-to-motion generation tasks, respectively, including mathematical formulations, training details and evalutation metrics chosen. Finally, we draw conclusion in chapter 5 and present results of the proposed methods and a discussion on the findings and future directions.

[1] *More information about the dataset and how to obtain it can be found here.*

Background

1 Task Definition

To begin with, we introduce the human motion data representations, then formally define the two tasks of interest in this work, namely motion-to-text captioning and text-to-motion generation.

1.1 Motion Data

Human motion data is typically represented through a simplified motion human skeletons structure, with a set of connected joints. That can be categorize into two main types: *keypoint-based* or *position-based* and *rotation-based* representations [15, 16]. We can transit from one representation to another using forward kinematics and inverse kinematics. Other representations exist, such as the *velocity-based* representation, using change of joints positions or rotations between consecutive frames.

Rotation-based With the *rotation-based* formalism, human pose is represented by joint rotations mostly in $SO(3)$[1], which can be expressed in various formats, such as Euler angles, axis-angle, or quaternions. A widely use rotation-based representation is the parametric model SMPL [17]. It employs a set of pose parameters $\theta \in \mathbb{R}^{3\times J}$ and shape parameters $\beta \in \mathbb{R}^{10}$ to define the human body mesh, where J is the number of joints. Pose parameters θ encode the relative rotations of each joint, while shape parameters β process body shape configurations. From the pose and shape parameters, the model generates a mesh representation of the human body. Other models derived from SMPL, such as SMPL-X [18] and SMPLH [19], extend the original model to include additional joints and body parts, such as the face, hands, and feet, providing a more detailed representation of human motion.

Position-based The *position-based* representation describes, for each of the T frames, human motion as a set of 3D coordinates for each of the J joints. In other words, for each frame $m_t \in \mathbb{R}^{J\times 3}$, $m_{t,j} = (x_{t,j}, y_{t,j}, z_{t,j})$ represents the spatial configuration of joint

[1] The special orthogonal group in three dimensions

j at time step t. Then a motion sequence $M \in \mathbb{R}^{T \times J \times 3}$ of length T is represented as a tensor $M = [m_1, m_2, \ldots, m_T]$. In this work, we focus on the position-based representation, which is the one used ont he HumanML3D dataset [14]. In this case, the representation use $J = 22$ joints, and the motion sequence length T varies from x to x frames with an average legnth of x frames.

1.2 Motion Captioning

The original task of motion-to-text captioning aims to model the conditional distribution of natural language descriptions given an input motion sequence, and to generate a caption that faithfully reflects the semantic content of the motion. This task is similar to image or video captioning, where the goal is to generate a textual description of visual content. These tasks could easily be separated into two sub-tasks: (1) extracting visual features from the input and (2) generating a textual description based on the extracted features. Several methods have been proposed to address these captioning tasks, based on the combination of an image encoder, usually a CNN for the first sub-task and a text decoder for the second sub-task. For the text decoder, RNNs were widely used in the early works [20, 21], then LSTMs were introduced [22, 23, 24] and [1, 25] proposed to use those combination video captioning seen as a sequence-to-sequence task, and showed the effectiveness of LSTMs in capturing temporal dependencies in video sequences. Afterwards, Transformers [26] have been introduced in the field of text generation, and have shown to outperform RNNs and LSTMs in various NLP tasks, such as machine translation, text summarization, and question answering. Therefore, transformer achitecture has been widely adopted in image and video captioning tasks [27, 28, 29]. Despite the similarities between motion captioning and image/video captioning, motion captioning presents unique challenges due to the inherent complexity of human motion.

1.3 Motion Generation

For the human motion generation task, methods are classified into two main categories: *regression-based* methods and *generative* methods.

2 Attention mechanism

The first definition of Transformer architecture and self-attention mechanism is presented in the following paper [26]. The following section provides a brief overview of the mechanism presented on the previous paper, which is used in this work for both motion captioning and motion generation tasks.

Self-attention Initially, for a sequene of embedding $(w_1, w_2, \ldots, w_l)$ where $w_i \in \mathbb{R}^{d_{\text{model}}}$, the first step of this mechanism is to linearly map the embedding w_i into query $q_i \in \mathbb{R}^{d_k}$,

key $k_i \in \mathbb{R}^{d_k}$, and value $v_i \in \mathbb{R}^{d_v}$ representations. Then the dot product is computed between each queries and all keys scaled by the square root of the key dimension $\sqrt{d_k}$, which measures the similarity of each key to the corresponding query. Then a softmax function is applied to the result to compute a weighted sum of the value. In practice, the attention mechanism is compute simultaneously for all queries, keys, and values packed together in matrices $Q, K, V \in \mathbb{R}^{l \times d_{\text{model}}}$, where l is the length of the input sequence, and d_{model} the dimension of the model. The scaled dot-product attention is then computed as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \tag{2.1}$$

Moreover, in Transformer architectures, this operation is performed in parallel over h heads:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O, \tag{2.2}$$

where each head is defined as: $\text{head}_i = \text{Attention}(QW_Q^{(i)}, KW_K^{(i)}, VW_V^{(i)})$, and $W_O \in \mathbb{R}^{hd_k \times d_{\text{model}}}$ is a learnable linear output projection.

Cross-attention To condition text generation on motion information, the decoder attends to the output of the motion encoder through a cross-attention mechanism. Let $H \in \mathbb{R}^{T \times d_{\text{model}}}$ denote the final output of the motion encoder. In the cross-attention sublayer, the decoder hidden states serve as queries, while the encoder outputs are used as keys and values:

$$Q = YW_Q, \quad K = HW_K, \quad V = HW_V,$$

where $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ are learnable projection matrices.

The attention operation is computed using scaled dot-product attention defined in (**??**). This mechanism enables the decoder to dynamically focus on relevant motion features when generating each output token.

3 Graph Neural Networks

Graph Neural Networks (GNNs) are a class of neural network architectures designed to operate on graph-structured data. Unlike conventional neural networks, which typically assume a regular structure such as a sequence or a grid, GNNs can directly model relationships between entities represented as nodes and edges. A graph can be formally defined as $G = (V, E)$, where V denotes the set of nodes and $E \subseteq V \times V$ denotes the set of edges connecting pairs of nodes. Each node $v_i \in V$ can be associated with a feature vector $\mathbf{h}_i$, while edges can optionally contain additional features describing the relationships between connected nodes.

The central principle underlying GNNs is that the representation of a node should incorporate information from its local neighborhood. Through successive graph convolution

or message-passing operations, information is propagated across the graph, allowing the network to learn representations that capture both the features of individual nodes and the structural relationships between them. This property makes GNNs particularly suitable for applications involving inherently relational data, such as human skeletons, where joints can be represented as nodes and anatomical connections as edges.

3.1 Message Passing

Message passing is a fundamental mechanism used by GNNs to propagate information between connected nodes. At each layer of a GNN, every node aggregates information from its neighboring nodes and combines this information with its own representation to produce an updated node representation. Let $\mathcal{N}(v_i)$ denote the set of neighboring nodes of node v_i. A generic message-passing layer can be expressed as

$$\mathbf{m}_i^{(l)} = \text{AGGREGATE}^{(l)}\left(\left\{\mathbf{h}_j^{(l)} \,\middle|\, v_j \in \mathcal{N}(v_i)\right\}\right), \tag{2.3}$$

where $\mathbf{h}_j^{(l)}$ represents the feature representation of node v_j at layer l, and $\mathbf{m}_i^{(l)}$ denotes the message aggregated for node v_i.

The aggregated message is subsequently combined with the current representation of the node through an update function:

$$\mathbf{h}_i^{(l+1)} = \text{UPDATE}^{(l)}\left(\mathbf{h}_i^{(l)}, \mathbf{m}_i^{(l)}\right). \tag{2.4}$$

A commonly used formulation combines the neighboring features through a learnable transformation followed by an aggregation operation:

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\mathbf{W}^{(l)}\,\text{AGGREGATE}\left(\left\{\mathbf{h}_j^{(l)} \,\middle|\, v_j \in \mathcal{N}(v_i)\right\}\right)\right), \tag{2.5}$$

where $\mathbf{W}^{(l)}$ is a learnable weight matrix and σ denotes a nonlinear activation function.

The aggregation function is typically designed to be permutation invariant, meaning that the result does not depend on the order in which neighboring nodes are presented. Common aggregation operations include summation, averaging, and maximization. This property is essential because the nodes in a graph do not possess an inherent ordering.

For a graph representing a human skeleton, each node can correspond to an individual body joint, while the edges represent anatomical connections between joints. Message passing therefore enables each joint to incorporate information from neighboring joints. Stacking multiple GNN layers progressively expands the receptive field of each node, allowing information to propagate across increasingly larger portions of the skeleton. Consequently, the resulting representations can capture both local relationships, such as those between adjacent joints, and higher-level structural dependencies across the entire body.

4 Diffusion models

There are mainly three types of diffusion models: *Score Matching with Langevin Dynamics* (SMLD) [30], *Denoising Diffusion Probabilistic Model* (DDPM) [9], and *Score-based Generative Model* (SGM) [31]. However, in this work, we only focus on the DDPM paradigm. As introduced in the paper by Ho et al.[9], diffusion process corresponds to two Markov chains that deconstruc data with defined noise and reconstruct the desired samples from the noise.

The forward chain gradually adds Gaussian noise, to a data sample x_0 of distribution $q(x_0)$, over a series of time steps $(t = 1, \ldots, T)$, resulting in a sequence of latent variables $(x_1, \ldots, x_T)$. In order to end up with a clean standard normal distribution at the final time step T, the forward process is governed by a variance schedule $(\beta_1, \ldots, \beta_T)$, and is defined as follows:

$$q(x_{1:T}|x_0) = \prod_{t=1}^{T} q(x_t|x_{t-1}), q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t\mathbf{I}) \tag{2.6}$$

Additionally, instead of sampling step-by-step iteratively, the paper leverages the property that $q(x_t|x_0)$ can be calculated directly in closed form at any arbitrary timestep t. Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, the distribution becomes:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I}) \tag{2.7}$$

and, any sample $x_t \sim q(x_t|x_0)$ can be expressed as:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon \quad \text{where } \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

On the other hands, the reverse process aims to reconstruct the original data sample x_0 from the noisy sample x_T by learning the parameterized Gaussian transition $p_\theta(x_{t-1}|x_t)$ at each time step t. More specifically, from a pure noise $x_T \sim \mathcal{N}(0, \mathbf{I})$ the reverse process is defined as:

$$p_\theta(x_0) = \int p_\theta(x_{0:T}) dx_{1:T}, \tag{2.8}$$

where the joint distribution $p_\theta(x_{0:T})$ is defined as:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1}|x_t),$$

and the transition distribution $p_\theta(x_{t-1}|x_t)$ is modeled as a Gaussian distribution:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

with $\mu_\theta(x_t, t)$ and $\Sigma_\theta(x_t, t)$ are the mean and covariance of the Gaussian distribution, which are parameterized by a neural network with learnable parameters θ. The transition sample

x_{t-1} is then generated by sampling from this Gaussian distribution, which is conditioned on the current noisy sample x_t and the time step t,

$$x_{t-1} = \mu_\theta(x_t, t) + \Sigma_\theta(x_t, t)\epsilon \quad \text{where } \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

.
The training objective and loss function of diffusion models are discussed in Section 2.1 and Section 2.2, respectively.

Motion-to-Text

1 Proposed architecture

1.1 Overview

The architecture proposed can be viewed as an encoder-decoder architecture, where the encoder is a concatenation of a spatio-temporal transformer encoder followed by a temporal transformer encoder. The first sublayer seize spatial interactions between joints in intra-frame through spatial attention. Subsequently, the temporal transformer encoder captures inter-frame temporal dependencies through temporal convolution. Moreover, in order to obtain additional temporal dependencies, the spatio-temporal encoder is followed by a temporal transformer encoder, which captures long-term temporal dependencies through temporal attention. Afterwards, a temporal Transformer decoder is used to generate a textual description of the motion sequence based on the spatio-temporal features previously provided by the encoder.

The following section provides a detailed description of this encoder-decoder architecture.

1.2 Spatio-temporal Transformer

Based on Chen et al. [10] proposed architecture, we simple transposed it to a motion task, where the human body of each frame in the motion is considered as a graph. The nodes correspond to the joints of the human body and the edges represent the connections between them. The spatial self-attention is still applied to the spatial dimension of the

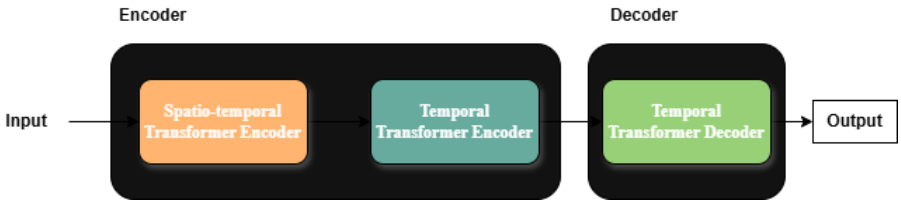


Fig. 3.1: Spatio-temporal Transformer architecture

motion data, while the temporal convolution is applied to the temporal dimension in order to capture temporal dependecies.

1.2.1 Spatial Attention

For each joint i in a given frame t, from embeddings $h_{t,i} \in \mathbb{R}^{d_{\text{model}}}$ query $q_{t,i} \in \mathbb{R}^{d_k}$, key $k_{t,i} \in \mathbb{R}^{d_k}$ and value $v_{t,i} \in \mathbb{R}^{d_v}$ representations are computed through linear projections, as follows:

$$q_{t,i} = W_q \cdot h_{t,i}, k_{t,i} = W_k \cdot h_{t,i}, v_{t,i} = W_v \cdot h_{t,i}, \tag{3.1}$$

Then the attention score is computed with respect to all other joints j in the same frame, where $j \in [\![1; J]\!]$, by applying the scaled dot-product between $q_{t,i}$ and $k_{t,j}$. It representsthe spatial message $m_{t,ij} = q_{t,i} \cdot k_{t,j}^T$ passing from joint j to joint i at time step t. In order to obtain the final attention score $a_{t,i,j}$ for each joint j with respect to joint i, the softmax function is applied to the spatial message $m_{t,ij}$ scaled by the square root of the key dimension $\sqrt{d_k}$, as follows:

$$a_{t,ij} = \text{softmax}\left(\frac{m_{t,ij}}{\sqrt{d_k}}\right). \tag{3.2}$$

The final output of the spatial attention for joint i at time step t is then computed as a weighted sum of the value representations $v_{t,j}$ of all joints j, using the attention scores $a_{t,ij}$:

$$head_{t,i} = \sum_{j=1}^{J} a_{t,ij} \cdot v_{t,j}. \tag{3.3}$$

By applying this spatial attention mechanism to H heads those are concatenated and linearly transformed, as follows:

$$\text{MultiHead}_{t,i} = \text{Concat}(head_{t,i}^{(1)}, \ldots, head_{t,i}^{(H)})W_O, \tag{3.4}$$

Finally, applied to all joints in the frame, we obtain a set of updated joint representations that capture the spatial dependencies between joints within the same frame.

1.2.2 Temporal Convolution

Given an input of shape $(B, T, J, d_{\text{model}})$, where B is the batch size, T is the number of frames, J is the number of joints, and d_{model} is the feature dimension, a temporal convolution with kernel size K is applied along the temporal dimension T for each joint $i \in [\![1; J]\!]$ independently. This operation captures temporal dependencies between frames, and represent trajectory of each joint during the sequence.

A skip connection is added to the output of the temporal convolution, followed by a layer normalization step.

1.3 Temporal Transformer

Temporal Transformer as defined in [10] is an encoder decoder architecture. The encoder is composed of a temporal self-attention sub-layer and a position-wise feed-forward network, while the decoder consists of a temporal self-attention sub-layer, a cross-attention sub-layer, and a position-wise feed-forward network. Each sub-layer is followed by a residual connection and layer normalization.

The temporal encoder mechanism uses multihead self-attention to capture long-term temporal dependencies between frames in the motion sequence, independently for each joint. A simple full connected network is used as the second sub-layer.

1.4 Transfer learning

Employing a pre-trained language model decoder offers several advantages. First, it substantially reduces the amount of paired motion and text data required for effective training. Second, it improves training stability and generalization by leveraging robust linguistic representations learned during pre-training. Finally, transfer learning allows the model to concentrate on learning the alignment between motion representations and language, rather than learning language generation from scratch, resulting in more fluent and semantically accurate text outputs.

The text decoder used in this work is based on the T5 pre-trained Transformer language model, following the encoder-decoder architecture introduced by [13]. This model reformulates all NLP tasks into a unified text-to-text paradigm, making it particularly suitable for conditional text generation from motion representations.

Thereby, the decoder is initialized from a pre-trained T5 model and operates in an autoregressive manner, i.e. predicts the next value in the output sequence based on the previous values. In this context, at each time step, the decoder generates a text token conditioned on the output of the motion encoder and on the previously generated tokens. By leveraging large-scale pre-training, the decoder benefits from strong linguistic priors, such as syntactic structure and semantic consistency, which would be difficult to learn from limited specific data alone.

Moreover, as the motion encoder, the decoder consists of a stack of $N = 6$ layers, each composed of three sub-layers: a self-attention mechanism and a cross-attention mechanism, and a position-wise feed-forward network. Likewise, each sub-layer is followed by a residual connection and layer normalization.

2 Experiments

All architectures are implemented in PyTorch. The model parameters are optimized using the Adam optimizer. The learning rate is set to $\eta = 10^{-4}$, and training is performed using batches of 16 sequences of approximately same legnth, padded if needed. To improve

training stability, gradient clipping is applied, and the learning rate may be decayed during training according to a predefined schedule. As description could be of different lengths, we define a maximum length of 50 tokens for the generated text, which corresponds to the mean length of the training sequences. Sequences shorter than this length are padded with a special token. All components of the model, including the motion encoder and the text decoder, are trained end-to-end.

2.1 Loss functions

Cross entropy loss The model is trained to maximize the likelihood of the ground-truth text sequence conditioned on the input motion representation. Given a target sequence of tokens $\mathbf{y_{gt}} = (y_1, y_2, \ldots, y_j)$, the training objective is defined using the cross-entropy loss. At each decoding step t, the model predicts a probability distribution $p(y_t \mid y_{<t}, H)$ over the vocabulary, where H denotes the output of the motion encoder. The loss is computed as:

$$\mathcal{L}_{\text{CE}} = -\sum_{t=1}^{T_t} \log p(y_t \mid y_{<t}, H). \tag{3.5}$$

Minimizing this loss encourages the model to assign high probability to the correct target tokens at each time step.

Contrastive learning Contrastive learning is a method representative of self-supervised learning. Radford et al. [32] proposed a contrastive learning approach to learn a joint embedding space for images and text, known as CLIP (Contrastive Language-Image Pre-training). It aims to maximize the similarity between an image embedding and its corresponding textual description embedding, while minimizing the similarity between an image embedding and other non-matching descriptions embeddings.

In this work, we adopt the same approach to learn a joint embedding space for motion and text, where the motion encoder and the text decoder are trained to produce embeddings that are close for matching motion-text pairs and far apart for non-matching pairs. Let's consider a batch of B paired motion and text samples embeddings, denoted as $\{(m_i, t_i)\}_{i=1}^{B}$, where m_i is the motion representation and t_i is the corresponding text description. The cosine similarity between each motion m_i and text t_j embeddings is computed to end up with a matrix of similarities $S \in \mathbb{R}^{B \times B}$. That way, the diagonal elements of S $\{(s_{i,i})\}_{i=1}^{B}$ are the similarities between matching motion-text pairs, while the off-diagonal elements $\{(s_{i,j})\}_{i \neq j}$ represent the similarities between non-matching pairs. Now the goal is to maximize the diagonal elements of S while minimizing the off-diagonal elements. Moreover, in order to influence the sharpness of the distribution, a scale, controlled by a temperature hyperparameter τ, can be applied to the cosine similarities matrix.

Teacher forcing During training, teacher forcing is employed to improve convergence and stability. Specifically, the ground-truth token y_{t-1} is provided as input to the decoder

when predicting the token y_t, rather than using the model's own previous prediction. This allows the decoder to learn the correct conditional distributions efficiently, while avoiding the accumulation of errors during training.

2.2 Evaluation metrics

For the evaluation of the generated text, we use a set of widely adopted metrics in natural language processing, including BLEU, ROUGE, METEOR, and CIDEr.

Text-to-Motion

1 Proposed architecture

1.1 Overview

For the generation of human motion from text, we propose an architecture that leverages the generation capabilities of diffusion models [9] coupled with spatio-temporal features extracted from the previous transformers architectures 1.

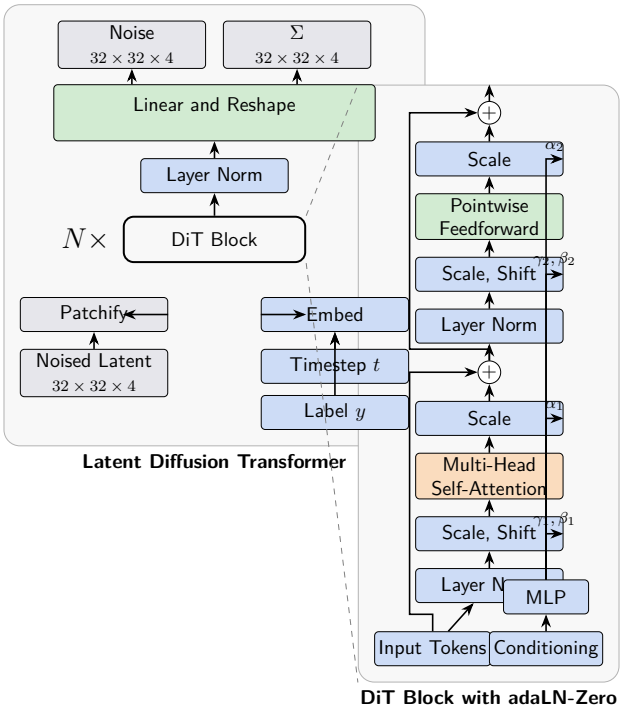


Fig. 4.1: Diffusion Transformer (DiT) architecture, reproduced from Peebles & Xie [33].

1.2 Spatio-temporal Diffusion Transformer

As presented in Section 4, the diffusion process consists of two Markov chain that deconstructs data with defined noise and reconstructs the desired samples from the noise. The following section describes our adaptation of the Diffusion Transformer (DiT) architecture proposed by Peebles et al. [33].

The input to Spatio-temporal Diffusion Transformer (StDiT) is a motion sequence $\mathbf{x}$ of T frames, represented as a tensor of shape $(T, J, 3)$. The motion sequence is first transformed into a latent representation $\mathbf{z}$ using a feed foward network, resulting in a tensor of shape (T, J, d_{model}), where d_{model} is the dimension of the latent space. Following the embedding of the motion sequence, we apply a positional encoding to the latent representation to capture the temporal information of the motion sequence and then pass it through a series of StDiT blocks. Additionally, alongside the noised input, the model processes conditional information in the form of a text description c and noise timestep t.
From the original paper, the DiT block is composed of an adaptative layer normalization (adaLN-Zero) followed by a residual connection and a layer normalization before the multihead self-attention and the pointwise feedforward layer. We decided to keep it that way, as it shown to be effective in the original paper [34, 33], and we believe it will help stabilize the training of our model. Moreover, we propose to replace the multihead self-attention mechanism in the DiT block with the proposed spatio-temporal attention block presented in Section 1.2.1.

2 Experiments

2.1 Training process

We train the StDiT model using the Adam optimizer [35], with $\beta_1 = 0.9$ and $\beta_2 = 0.999$. The learning rate is set to 10^{-4} with a weight decay of 10^{-5}. During training, data augmentation is initially applied to the training set; however, we restrict the augmentation strategy to horizontal flipping, as this is the only augmentation considered to preserve the semantic characteristics of the underlying motion.

2.2 Training Objective

The model is trained using the mean squared error (MSE) loss, which minimizes the discrepancy between the noise predicted by the model and the noise sampled during the forward diffusion process. Specifically, the objective encourages the model to accurately estimate the noise component added to each training sample, thereby facilitating the subsequent denoising process during generation.

2.3 Evaluation metrics

To evaluate the quality of the generated human motion, we employ several widely adopted metrics in the field of human motion generation. These metrics include the mean squared error (MSE), Fréchet Inception Distance (FID), and Mean Per Joint Position Error (MPJPE). Together, these metrics provide complementary measures of the accuracy and overall quality of the generated motion.

Results

1 Results Analysis

Initially, the tokenizer exhibited several decoding issues, notably incorrect subword combinations such as *"sclimbing"* or *"salimbs"*. In other cases, the decoding process entered repetitive loops, producing sequences with repeated tokens (e.g., *"legs legs legs ..."*) or hallucinated a lot and generated words in russian or german.

To mitigate these instabilities, several corrective measures were explored. First, motion features were normalized in order to reduce scale-related inconsistencies in the encoder outputs. Subsequently, the projection layer responsible for mapping the motion encoder representations to the T5 model embedding space was redesigned. Specifically, the original single linear layer was replaced with a three-layer multilayer perceptron (MLP), where each layer is followed by a ReLU activation and a dropout layer. This architectural modification improved the stability of the generated sequences and reduced repetitive decoding behavior.

2 Discussion

You probably found all the files from.

Training Error	**Testing Error**
0	∞

Tab. 5.1: The Results

Conclusion

Bibliography

[1] S. Venugopalan, M. Rohrbach, J. Donahue, R. Mooney, T. Darrell, and K. Saenko, "Sequence to sequence – video to text," 2015.

[2] X. Zhou, A. Arnab, S. Buch, S. Yan, A. Myers, X. Xiong, A. Nagrani, and C. Schmid, "Streaming dense video captioning," 2024.

[3] W. Mao, M. Liu, M. Salzmann, and H. Li, "Learning trajectory dependencies for human motion prediction," 2020.

[4] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2017.

[5] J. Martinez, M. J. Black, and J. Romero, "On human motion prediction using recurrent neural networks," 2017.

[6] K. Fragkiadaki, S. Levine, and J. Malik, "Recurrent network models for kinematic tracking," *CoRR*, vol. abs/1508.00271, 2015.

[7] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2022.

[8] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," 2014.

[9] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," 2020.

[10] W. Chen, F. Wang, and H. Sun, "S2tnet: Spatio-temporal transformer networks for trajectory prediction in autonomous driving," 2022.

[11] C. Yu, X. Ma, J. Ren, H. Zhao, and S. Yi, "Spatio-temporal graph transformer networks for pedestrian trajectory prediction," 2020.

[12] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," 2018.

[13] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," 2023.

[14] C. Guo, S. Zou, X. Zuo, S. Wang, W. Ji, X. Li, and L. Cheng, "Generating diverse and natural 3d human motions from text," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5142–5151, 2022.

[15] W. Zhu, X. Ma, D. Ro, H. Ci, J. Zhang, J. Shi, F. Gao, Q. Tian, and Y. Wang, "Human motion generation: A survey," 2023.

[16] B. Cheng, Z. Zhou, Y. Deng, S. Du, Z. Wang, Y. Wen, Y. Zhang, S. Ni, Y. Kong, L. Xie, and X. Yang, "Context-aware human motion generation: A comprehensive survey," *Design and Artificial Intelligence*, vol. 1, no. 1, p. 100007, 2025.

[17] M. Loper, N. Mahmood, J. Romero, G. Pons-Moll, and M. J. Black, "SMPL: A skinned multi-person linear model," *ACM Transactions on Graphics, (Proc. SIGGRAPH Asia)*, vol. 34, pp. 248:1–248:16, Oct. 2015.

[18] G. Pavlakos, V. Choutas, N. Ghorbani, T. Bolkart, A. A. A. Osman, D. Tzionas, and M. J. Black, "Expressive body capture: 3d hands, face, and body from a single image," in *Proceedings IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[19] J. Romero, D. Tzionas, and M. J. Black, "Embodied hands: Modeling and capturing hands and bodies together," *ACM Transactions on Graphics, (Proc. SIGGRAPH Asia)*, vol. 36, Nov. 2017.

[20] X. Chen and C. L. Zitnick, "Learning a recurrent visual representation for image caption generation," 2014.

[21] J. Mao, W. Xu, Y. Yang, J. Wang, Z. Huang, and A. Yuille, "Deep captioning with multimodal recurrent neural networks (m-rnn)," 2015.

[22] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan, "Show and tell: A neural image caption generator," 2015.

[23] K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhutdinov, R. Zemel, and Y. Bengio, "Show, attend and tell: Neural image caption generation with visual attention," 2016.

[24] S. J. Rennie, E. Marcheret, Y. Mroueh, J. Ross, and V. Goel, "Self-critical sequence training for image captioning," 2017.

[25] J. Donahue, L. A. Hendricks, M. Rohrbach, S. Venugopalan, S. Guadarrama, K. Saenko, and T. Darrell, "Long-term recurrent convolutional networks for visual recognition and description," 2016.

[26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023.

[27] Y. Luo, J. Ji, X. Sun, L. Cao, Y. Wu, F. Huang, C.-W. Lin, and R. Ji, "Dual-level collaborative transformer for image captioning," 2021.

[28] Y. Wang, J. Xu, and Y. Sun, "End-to-end transformer based model for image captioning," 2022.

[29] S. Herdade, A. Kappeler, K. Boakye, and J. Soares, "Image captioning: Transforming objects into words," 2020.

[30] Y. Song and S. Ermon, "Generative modeling by estimating gradients of the data distribution," 2020.

[31] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," 2021.

[32] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," 2021.

[33] W. Peebles and S. Xie, "Scalable diffusion models with transformers," 2023.

[34] D. Lian, D. Zhou, J. Feng, and X. Wang, "Scaling shifting your features: A new baseline for efficient model tuning," 2023.

[35] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017.

[36] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th annual meeting on association for computational linguistics*, pp. 311–318, Association for Computational Linguistics, 2002.

Appendix

Annexes

1 Markov Chain

A Markov chain is a stochastic process that describes a sequence of states in which the probability of transitioning to the next state depends only on the current state and not on the sequence of states that preceded it. This property is referred to as the *Markov property*. Formally, let $X_0, X_1, \ldots, X_T$ denote a sequence of random variables representing the states of the process. The Markov property can be expressed as

$$P(X_{t+1} \mid X_t, X_{t-1}, \ldots, X_0) = P(X_{t+1} \mid X_t). \tag{B.1}$$

A Markov chain is characterized by a state space S and a transition probability distribution. For a discrete state space, the transition probability from state i to state j is defined as

$$P_{ij} = P(X_{t+1} = j \mid X_t = i). \tag{B.2}$$

These transition probabilities can be represented by a transition matrix $\mathbf{P}$, whose elements satisfy

$$P_{ij} \geq 0, \qquad \sum_{j \in \mathcal{S}} P_{ij} = 1. \tag{B.3}$$

Consequently, each row of the transition matrix defines a probability distribution over the possible states at the next time step. The probability of transitioning from an initial state to a state after multiple steps can be obtained by raising the transition matrix to the corresponding power. In particular, the n-step transition probability is given by

$$P(X_{t+n} = j \mid X_t = i) = (\mathbf{P}^n)_{ij}. \tag{B.4}$$

Markov chains provide a useful mathematical framework for modeling sequential and stochastic processes. In the context of diffusion-based generative models, the forward diffusion process can be formulated as a Markov chain in which each state is obtained from the preceding state by progressively adding noise. This formulation allows the forward process to be decomposed into a sequence of simple transitions, which is subsequently exploited by the generative model to learn the reverse denoising process.

2 Description of Evaluation Metrics

This section describes the evaluation metrics used to assess the quality of the generated outputs. The selected metrics provide complementary measures of similarity between generated and reference outputs, with each metric capturing different aspects of the generated content.

BLEU The Bilingual Evaluation Understudy (BLEU) metric [36] is an automatic evaluation metric originally proposed for machine translation. It measures the similarity between a generated text and one or more reference texts based primarily on the precision of matching n-grams. BLEU considers n-grams of different orders, typically ranging from unigrams to four-grams, and combines their modified precisions using a geometric mean.

The modified precision for an n-gram is computed by clipping the number of occurrences in the generated text according to the maximum number of occurrences found in the reference texts. This prevents frequently repeated n-grams from artificially increasing the score. The BLEU score is then defined as

$$BLEU = BP \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right), \tag{B.5}$$

where p_n denotes the modified precision for n-grams of order n, w_n is the weight assigned to each n-gram order, and BP is the brevity penalty. The brevity penalty penalizes candidate texts that are substantially shorter than the reference texts.

BLEU scores generally range from 0 to 1, with higher values indicating greater n-gram overlap between the generated and reference texts. Although BLEU is effective at measuring lexical similarity, it may not fully capture semantic similarity when the generated text uses alternative wording or sentence structures.

ROUGE Recall-Oriented Understudy for Gisting Evaluation (ROUGE) [**?**] is a family of metrics commonly used to evaluate automatically generated text, particularly in text summarization. Unlike BLEU, which primarily emphasizes precision, ROUGE focuses on the degree to which information from the reference text is recovered by the generated text.

Several variants of ROUGE have been proposed. ROUGE-N measures the overlap of n-grams between the candidate and reference texts. For example, ROUGE-1 evaluates unigram overlap, while ROUGE-2 evaluates bigram overlap. The recall-oriented formulation of ROUGE-N can be expressed as

$$ROUGE\text{-}N = \frac{\sum_{g \in \text{Reference}} \min\left(\text{Count}_{\text{cand}}(g), \text{Count}_{\text{ref}}(g)\right)}{\sum_{g \in \text{Reference}} \text{Count}_{\text{ref}}(g)}, \tag{B.6}$$

where g represents an n-gram, and $\text{Count}_{\text{cand}}(g)$ and $\text{Count}_{\text{ref}}(g)$ represent the number of occurrences of g in the candidate and reference texts, respectively.

Another commonly used variant, ROUGE-L, is based on the longest common subsequence (LCS) between the generated and reference texts. ROUGE-L considers the sequential structure of the text and can therefore capture similarities that are not necessarily represented by contiguous n-grams. Higher ROUGE scores generally indicate greater overlap with the reference text.

METEOR The Metric for Evaluation of Translation with Explicit ORdering (METEOR) [**?**] evaluates the similarity between a generated text and a reference text by considering unigram matches. In contrast to metrics based exclusively on exact word overlap, METEOR can account for linguistic variations through stemming and synonym matching.

METEOR combines unigram precision and recall using their harmonic mean. Given unigram precision $\mathbf{P}$ and recall $\mathbf{R}$, the corresponding score can be expressed as

$$F_{\text{mean}} = \frac{10PR}{R + 9P}. \tag{B.7}$$

METEOR additionally incorporates a fragmentation penalty to account for differences in word ordering. Let m denote the number of matched unigrams and ch the number of contiguous matching chunks. The fragmentation penalty is defined as

$$Penalty = \gamma \left(\frac{ch}{m} \right)^{\beta}, \tag{B.8}$$

where γ and β are parameters controlling the magnitude and sensitivity of the penalty. The final METEOR score is then obtained as

$$METEOR = F_{\text{mean}}(1 - \text{Penalty}). \tag{B.9}$$

By incorporating both precision and recall, as well as synonym and stemming information, METEOR is generally more sensitive to semantic equivalence than metrics based solely on exact n-gram matching.

CIDEr Consensus-based Image Description Evaluation (CIDEr) [**?**] is an evaluation metric originally developed for image captioning. It measures the similarity between a generated description and a set of reference descriptions by evaluating their agreement in terms of TF-IDF-weighted n-grams. The use of TF-IDF weighting gives greater importance to n-grams that are informative and distinctive, while reducing the influence of common words.

For an n-gram g, the TF-IDF weight is defined as

$$w_g = \text{tf}_g \log \left(\frac{N}{\text{df}_g} \right), \tag{B.10}$$

where tf_g denotes the term frequency of g, N is the total number of reference descriptions, and df_g denotes the number of reference descriptions containing g.

The similarity between a candidate description and a reference description is then computed using the cosine similarity between their TF-IDF-weighted n-gram representations. CIDEr typically considers n-grams of orders 1 through 4, and the final score is obtained by averaging the similarity across these orders and across the available reference descriptions.

Compared with metrics based solely on exact word overlap, CIDEr places greater emphasis on content words and n-grams that are representative of the reference descriptions. Consequently, it is particularly suited to evaluating whether a generated description captures the salient information expressed by human references.